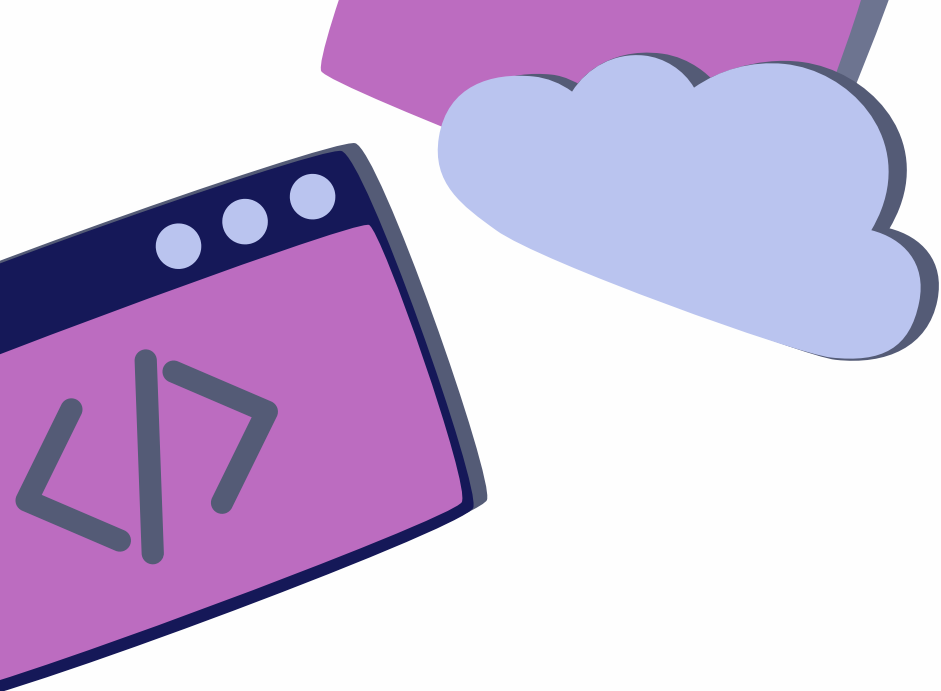




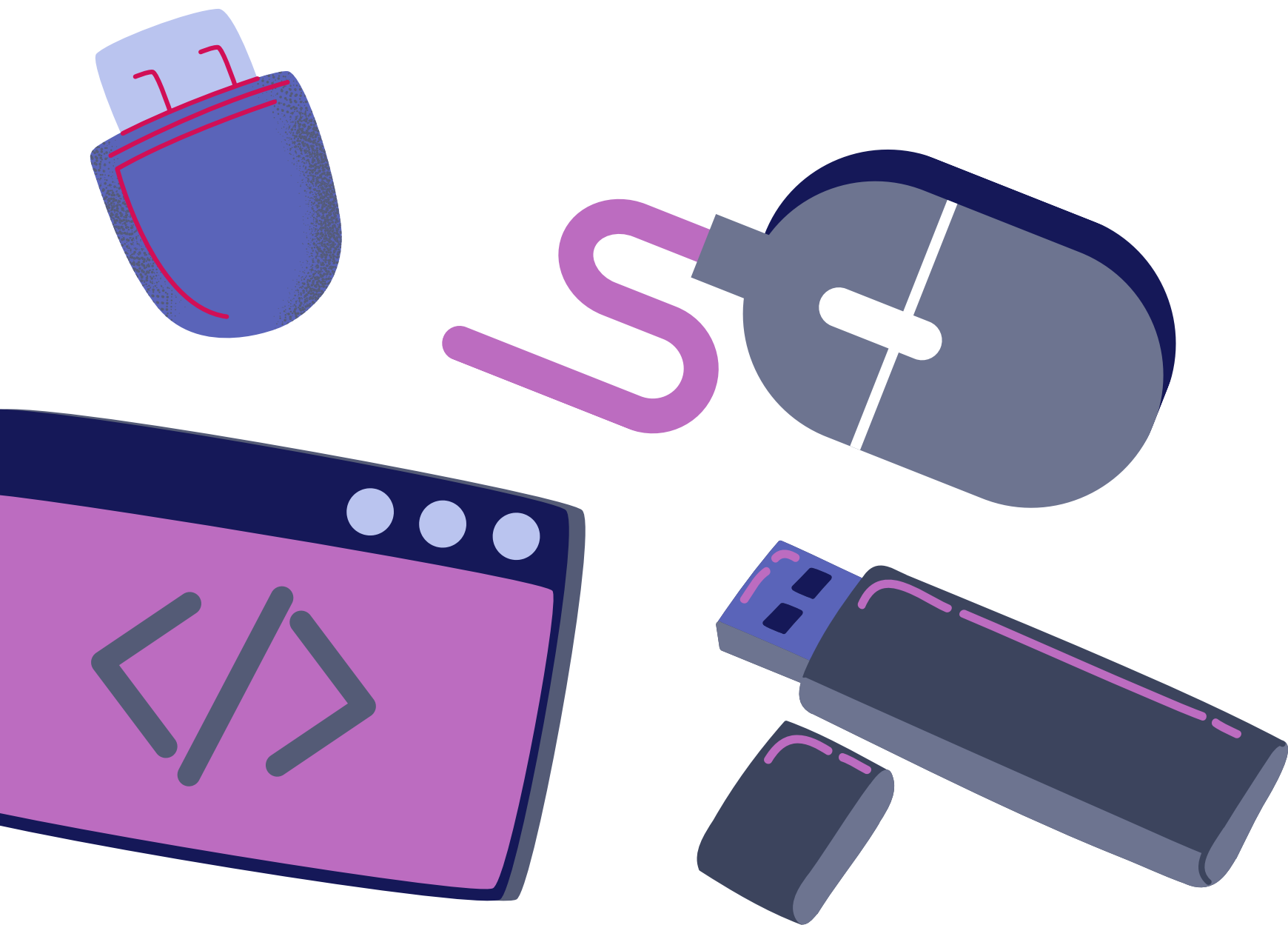
Ayudante: Myriam Alvarado

FUNDAMENTOS

de Programación



ÍNDICE



01. Introducción a Python

02. Tipos de Datos Primitivos

03. Definición y asignación de Variables

04. Tipos de Entradas y Salidas

05. Operadores y Métodos más usados

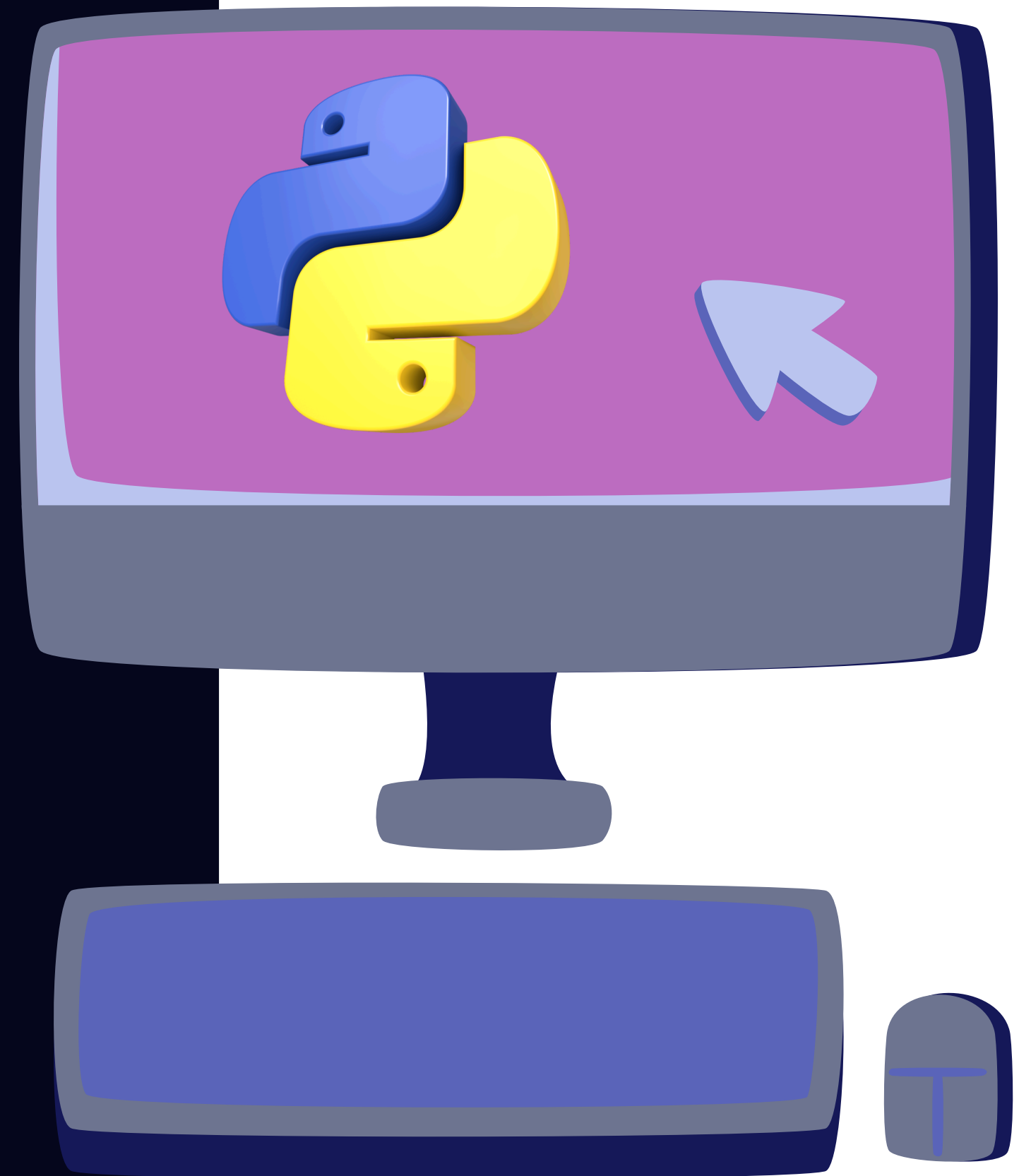
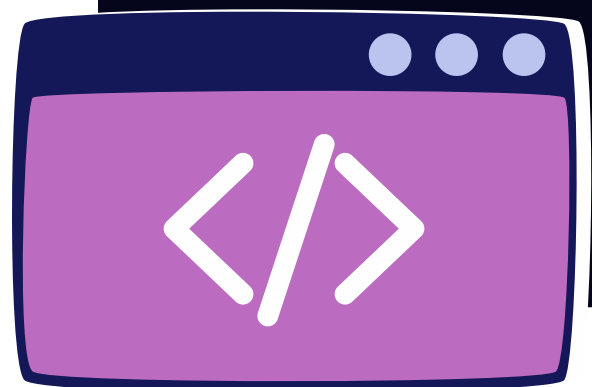
06. Definición de Funciones

07. Resumen y Recomendaciones

Introducción

El propósito específico de un programa puede resumirse en tres tareas:

1. Tomar información
2. Procesarla o transformarla
3. Producir nueva información





¿QUÉ ES PYTHON?

Python es un lenguaje de programación ampliamente utilizado en las aplicaciones web, el desarrollo de software, la ciencia de datos y el machine learning (ML).

Los desarrolladores utilizan Python porque es eficiente y fácil de aprender, además de que se puede ejecutar en muchas plataformas diferentes.

DATOS PRIMITIVOS

NÚMEROS ENTEROS

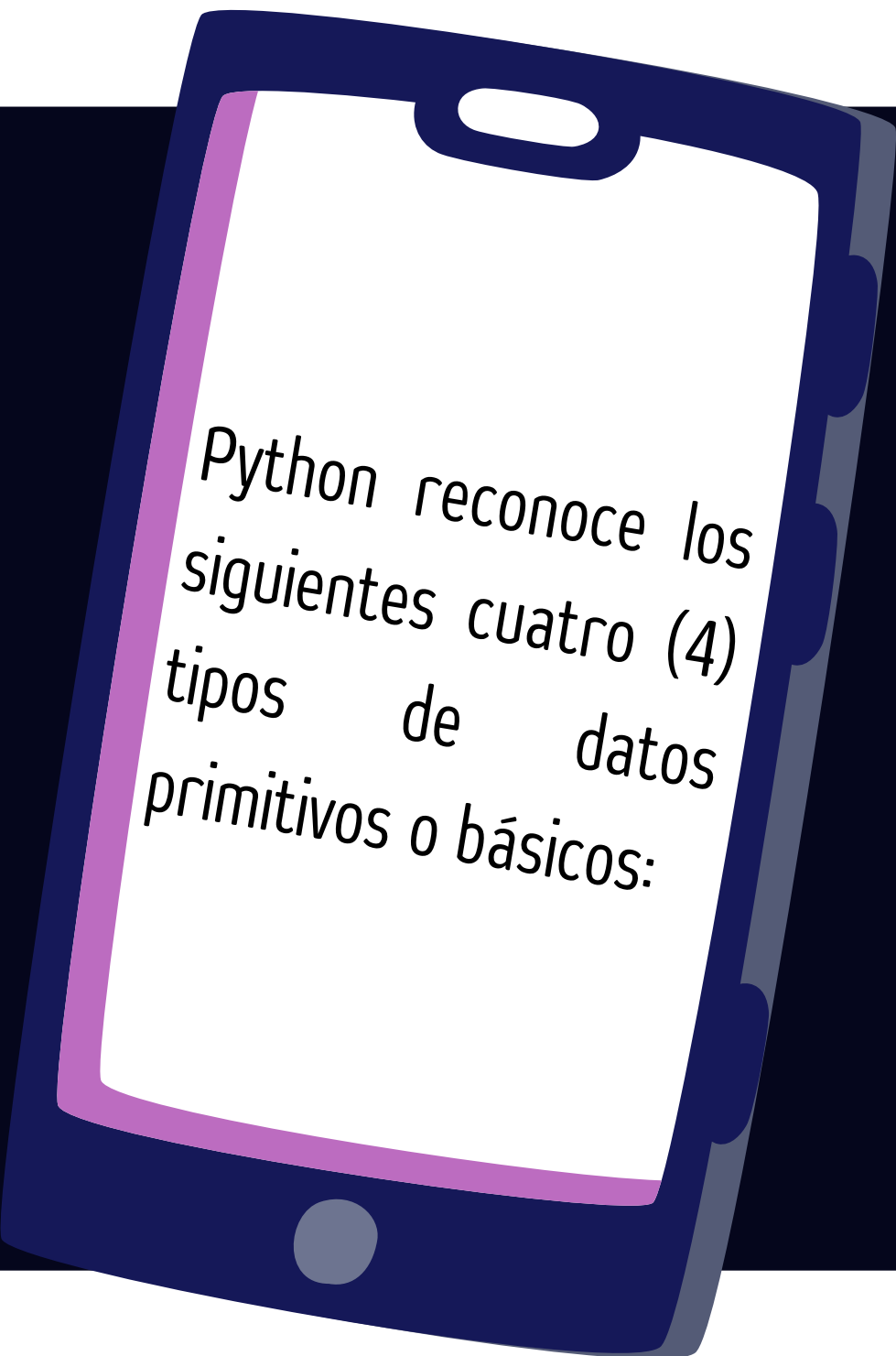
- Conocidos como tipo de dato int.
- Son números sin punto decimal.
- Ejemplos:

37
0
-128

TEXTOS

- Conocidos como tipo de dato str o string.
- Expresiones encerradas entre comillas simples o comillas dobles.
- Ejemplos:

'fundamentos'
"programación"
"2023"



Python reconoce los
siguientes cuatro (4)
tipos de datos
primitivos o básicos:

NÚMEROS DECIMALES

- Conocidos como tipo de dato float.
- Números con punto decimal o expresados en notación de 10.
- Ejemplos:

3.25
-0.028
2.4e-5

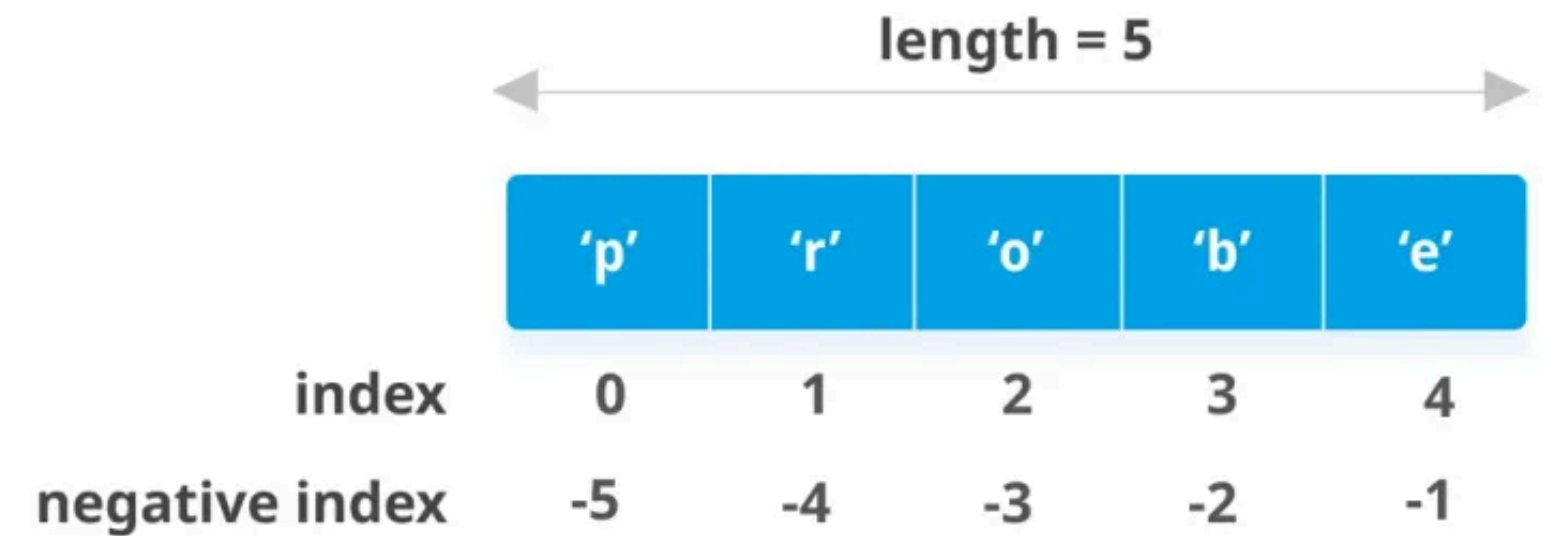
VALORES DE VERDAD

- Conocidos como tipo de dato bool o booleanos.

TRUE Representa al valor lógico verdadero.

FALSE Representa al valor lógico falso.

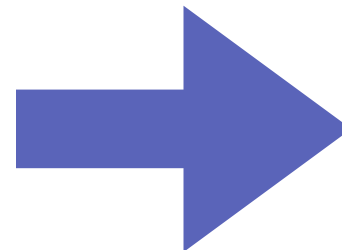
Muy IMPORTANTE



01

DE LOS TIPOS DE DATOS PRIMITIVOS COMO INT, FLOAT Y STR... SE PUEDE EXTRAER INFORMACIÓN

A través del Slicing e Index
¿De qué se tratan?



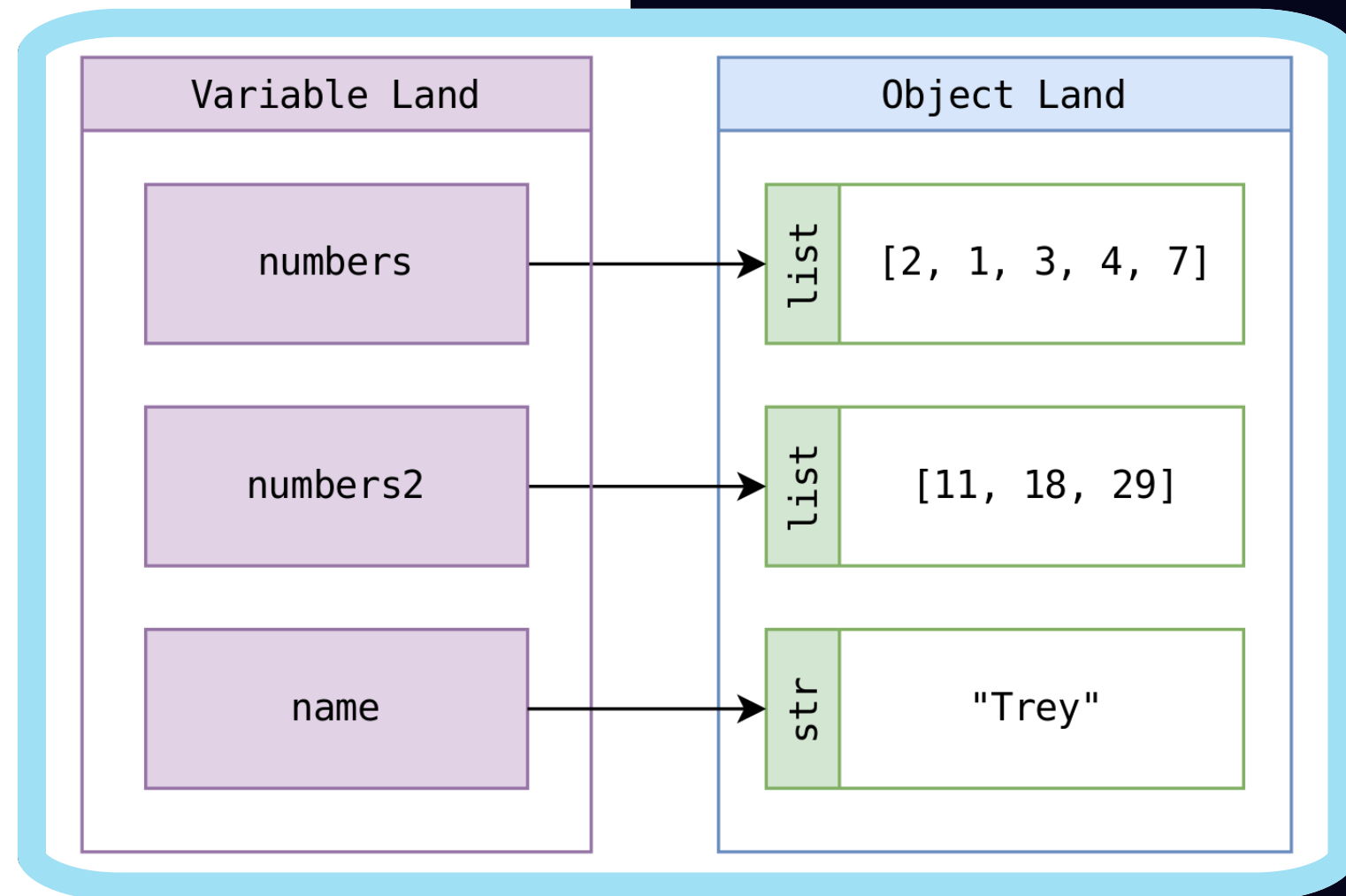
Primero veremos de qué se compone cada tipo de dato

02

PODEMOS TRANSFORMAR LOS TIPOS DE DATOS

IMPORTANTE!, CUANDO CONVERTIMOS FLOAT A INT SE PIERDE LA PARTE DECIMAL. ADEMÁS, ES POSIBLE CONVERTIR UN NÚMERO REPRESENTADO POR UN STR A INT O FLOAT SIEMPRE Y CUANDO EL STR NO CONTenga CARACTERES ESPECIALES.

<code>int()</code>	Convertir a entero	<code>int(x)</code>
<code>float()</code>	Convertir a decimal	<code>float(x)</code>
<code>str()</code>	Convertir a string	<code>str(x)</code>
<code>type()</code>	Consultar el tipo de dato	<code>type(x)</code>



Definición y asignación de Variables

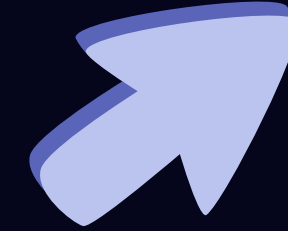
La forma de dar valor a una variable se realiza por asignaciones.

En la parte izquierda hay que situar un identificador válido y a la derecha la expresión a guardar.

identificador = expresión

KEYWORDS

```
Keywords in Python False None True and as assert async
await break class continue def del elif
else except finally for from global if
import in is lambda nonlocal not or
pass raise return try while with yield
```



Según la PEP 8 los nombres de las variables de Python deben de escribirse en snake_case. Además en deben de cumplir las siguientes características:

- Tienen que empezar por una letra-
- El uso de keywords como nombres está prohibido.
- Los nombres deben de ser descriptivos.
- Deben de estar en minúsculas y separando palabras por barras bajas '_'.

PEP 8

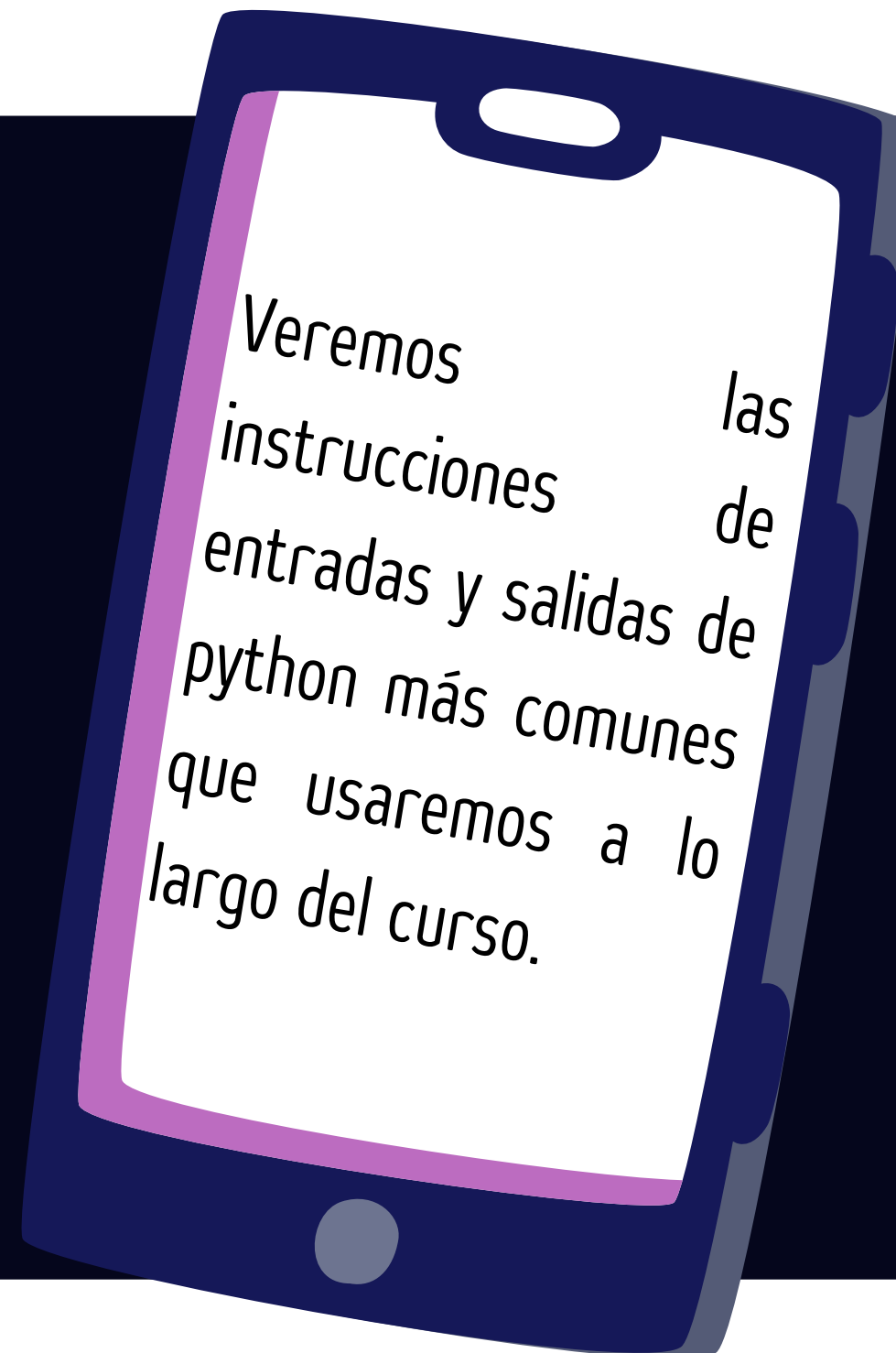


Guía de estilo para Python

TIPOS DE ENTRADAS Y SALIDAS

SALIDAS

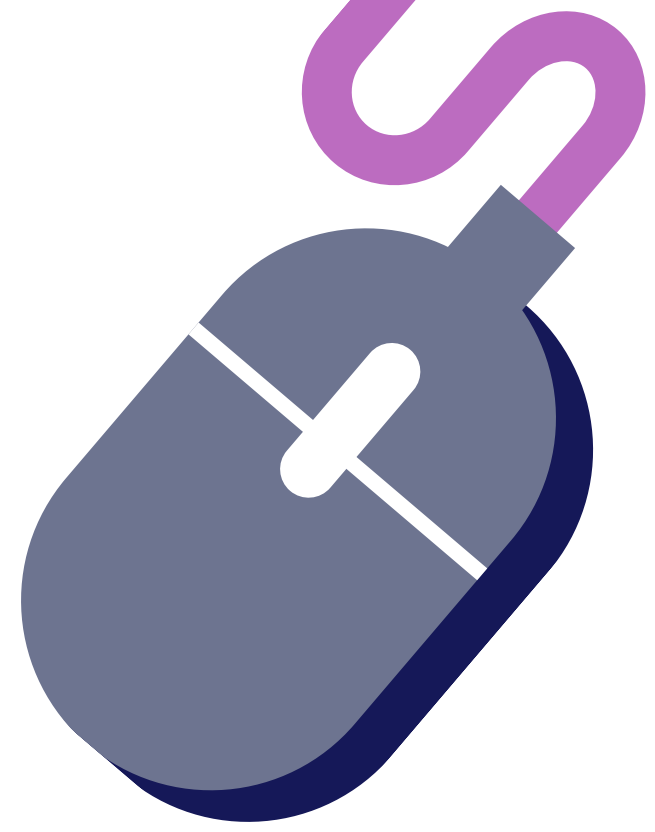
- La función `print()` nos permite mostrar información por consola como mensajes, números o valores de una variable. para su uso solo le pasamos en los argumentos lo que deseamos mostrar en consola. NO RETORNA.



ENTRADAS

- La función `input()` permite obtener información y guardarla en una variable para su posterior uso.
- Al momento de ejecutarse esta línea en la consola esperara que ingresemos el texto que necesitamos y demos un enter para continuar y guardar el valor capturado en una variable de programa. SI RETORNA.

Operadores y Métodos más utilizados



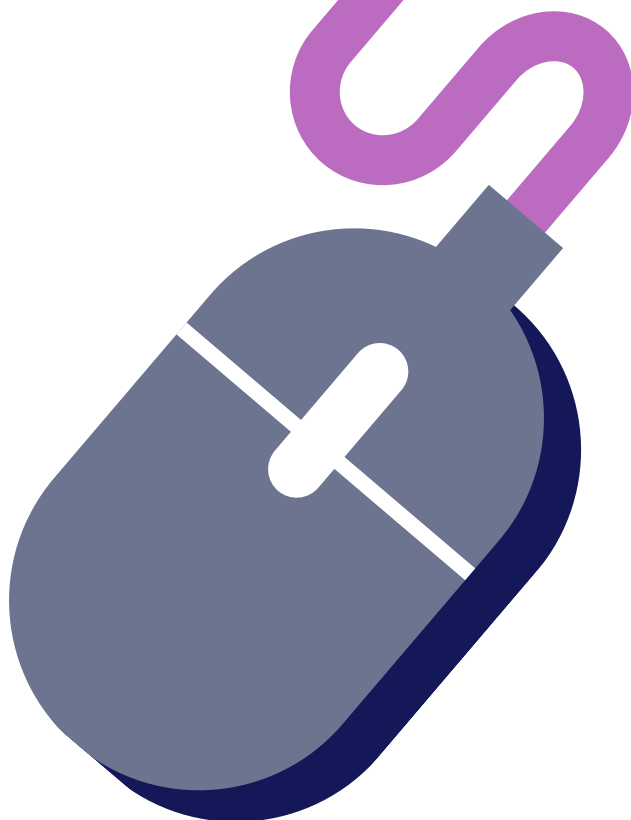
OPERACIONES BOOLEANAS

Operador	Descripción	Ejemplo de uso
<code>or</code>	Operación o	<code>x or y</code>
<code>and</code>	Operación y	<code>x and y</code>
<code>not</code>	Negación	<code>not x</code>

COMPARACIONES

Operador	Descripción	Ejemplo de uso
<code><</code>	Menor que	<code>a < b</code>
<code><=</code>	Menor o igual que	<code>a <= b</code>
<code>></code>	Mayor que	<code>a > b</code>
<code>>=</code>	Mayor o igual que	<code>a >= b</code>
<code>==</code>	Igual que	<code>a == b</code>
<code>!=</code>	Diferente de	<code>a != b</code>

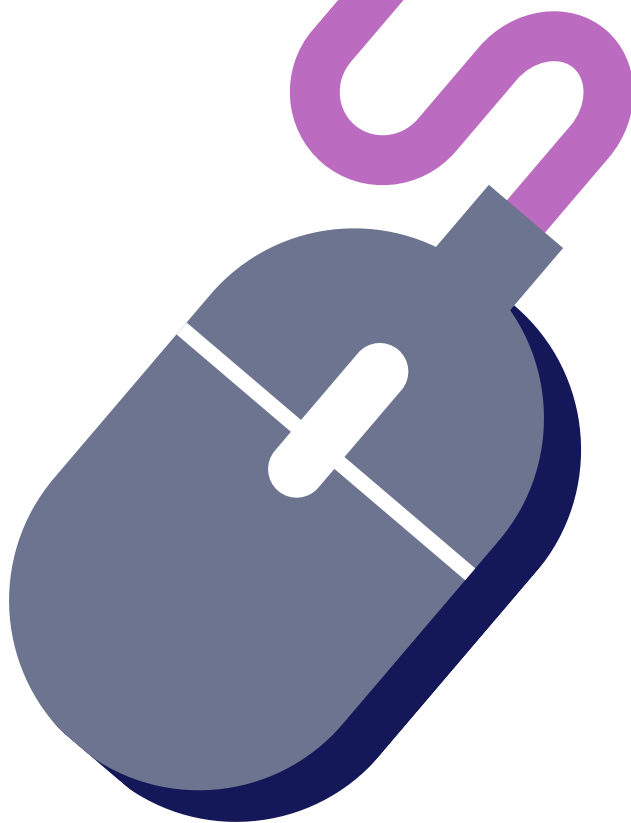
Operadores y Métodos más utilizados



OPERACIONES CON DATOS INT Y FLOAT

Operador	Descripción	Ejemplo de uso
+	Suma	<code>x + y</code>
-	Resta	<code>x - y</code>
*	Multiplicación	<code>x * y</code>
/	División	<code>x / y</code>
//	División entera	<code>x // y</code>
%	Módulo o residuo	<code>x % y</code>
**	Exponenciación	<code>x ** y</code>
-	Valor negativo	<code>-x</code>
+	Valor positivo	<code>+x</code>
<code>abs ()</code>	Valor absoluto	<code>abs (x)</code>

Operadores y Métodos más utilizados



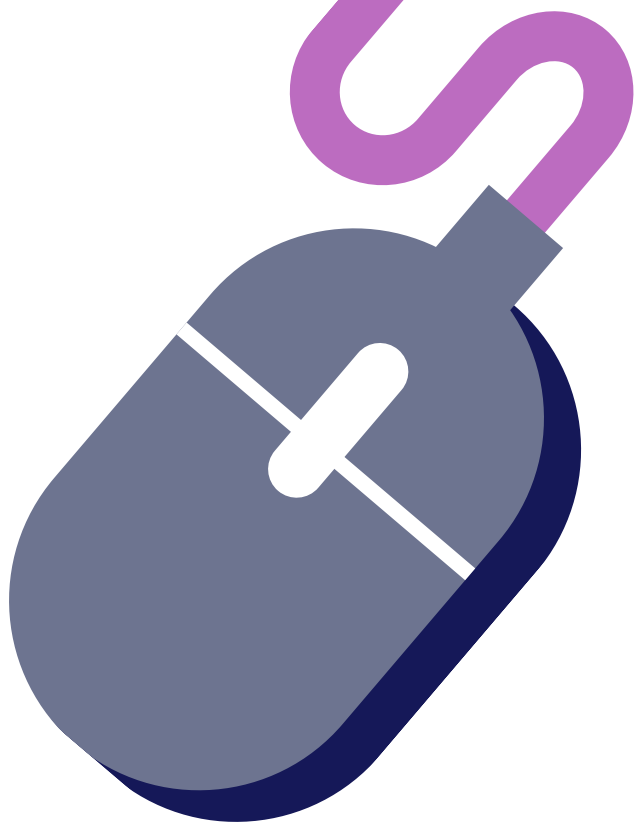
OPERACIONES CON STRINGS

Operador	Descripción	Ejemplo de uso
<code>in</code>	Retorna <code>True</code> si <code>x</code> está en <code>s</code>	<code>x in s</code>
<code>+</code>	Concatena (une) dos strings	<code>s + t</code>
<code>*</code>	Repetir un string <code>s</code> por <code>n</code> veces	<code>s * n, n * s</code>
<code>[i]</code>	Retorna el carácter de <code>s</code> en la posición <code>i</code>	<code>s[i]</code>
<code>[i:j]</code>	Retorna la subcadena de <code>s</code> desde <code>i</code> hasta <code>j - 1</code>	<code>s[i:j]</code>
<code>[i:j:k]</code>	Retorna la subcadena de <code>s</code> desde <code>i</code> hasta <code>j - 1</code> saltando <code>k - 1</code> elementos	<code>s[i:j:k]</code>
<code>len()</code>	Devuelve la cantidad de símbolos en <code>s</code>	<code>len(s)</code>
<code>min()</code>	Retorna el caracter con el menor valor	<code>min(s)</code>
<code>max()</code>	Retorna el caracter con el mayor valor	<code>max(s)</code>

Operadores y Métodos más utilizados

MÉTODOS CON STRINGS

Métodos	Descripción	Ejemplo de uso
<code>capitalize()</code>	Retorna una copia de <code>s</code> con el primer elemento de la cadena convertido a mayúscula	<code>s.capitalize()</code>
<code>count()</code>	Indica cuantas veces se repite <code>x</code> en <code>s</code>	<code>s.count(x)</code>
<code>endswith()</code>	Retorna <code>True</code> si <code>s</code> termina en <code>x</code>	<code>s.endswith(x)</code>
<code>startswith()</code>	Retorna <code>True</code> si <code>s</code> <u>empieza</u> con <code>x</code>	<code>s.startswith(x)</code>
<code>index()</code>	Retorna la posición de <code>x</code> en <code>s</code> o <code>ERROR</code> si no la encuentra	<code>s.index(x)</code>
<code>isalnum()</code>	Retorna <code>True</code> si <code>s</code> está compuesta únicamente por símbolos alfanuméricos (letras y números)	<code>s.isalnum()</code>
<code>isalpha()</code>	Similar a <code>isalnum()</code>	<code>s.isalpha()</code>
<code>isdigit()</code>	Retorna <code>True</code> si <code>s</code> está compuesta únicamente de números	<code>s.isdigit()</code>
<code>isnumeric()</code>	Similar a <code>isdigit()</code>	<code>s.isnumeric()</code>
<code>isdecimal()</code>	Similar a <code>isdigit()</code>	<code>s.isdecimal()</code>
<code>islower()</code>	Retorna <code>True</code> si <code>s</code> está compuesta únicamente de letras minúsculas	<code>s.islower()</code>
<code>isupper()</code>	Retorna <code>True</code> si <code>s</code> está compuesta únicamente de letras mayúsculas	<code>s.isupper()</code>
<code>lower()</code>	Retorna una copia de <code>s</code> con todo el texto convertido a minúsculas	<code>s.lower()</code>
<code>upper()</code>	Retorna una copia de <code>s</code> con todo el texto convertido a mayúsculas	<code>s.upper()</code>

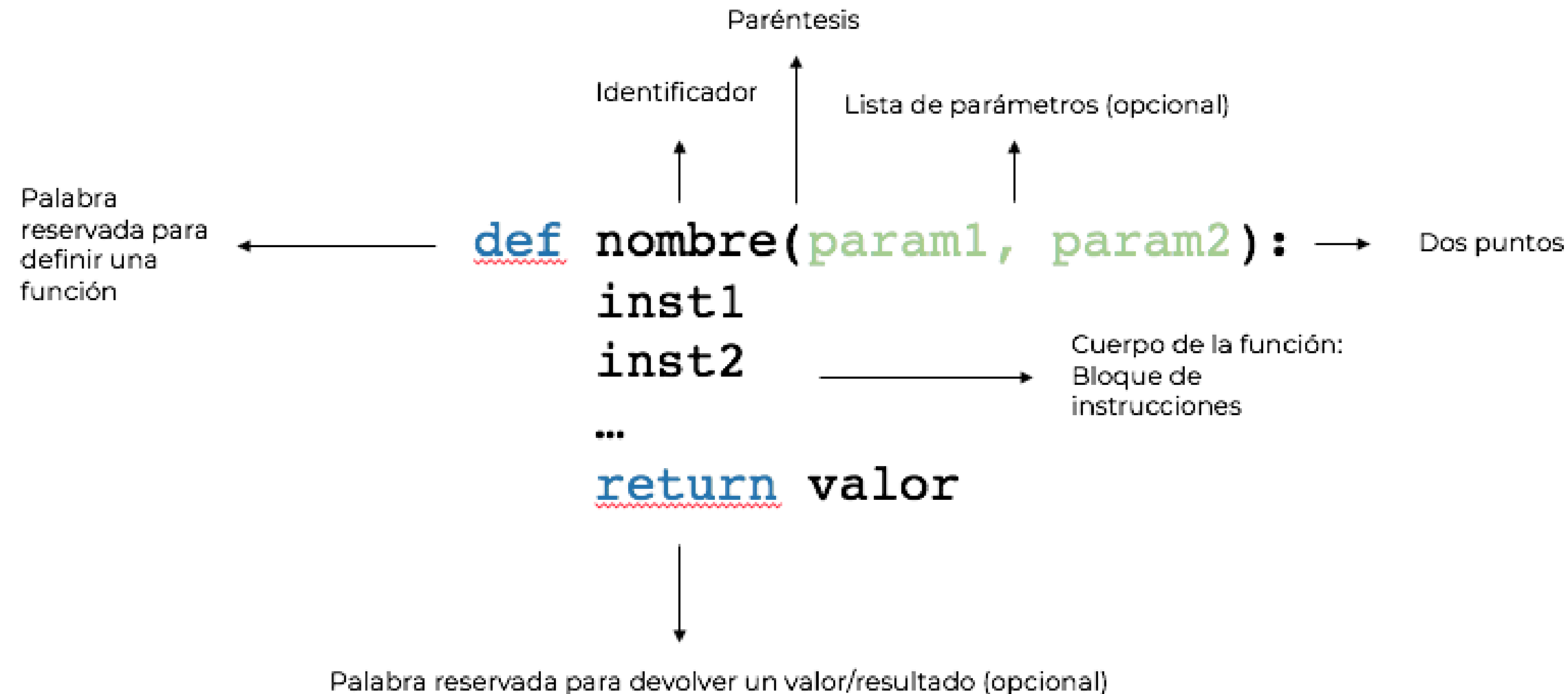


DEFINICIÓN DE FUNCIONES

Una función es un grupo de instrucciones que constituyen una unidad lógica del programa y resuelven un problema muy concreto.

Las funciones en Python tienen un doble objetivo:

- Dividir y organizar el código en partes más sencillas.
- Encapsular el código que se repite a lo largo de un programa para ser reutilizado.



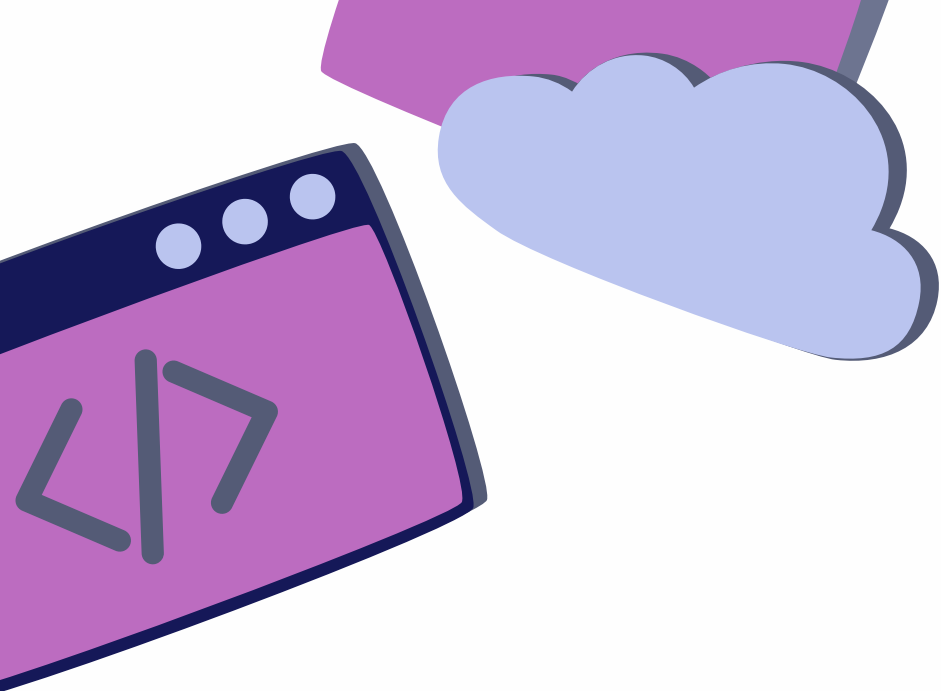
DEFINIR TUS PROPIAS FUNCIONES PARA ESTRUCTURAR EL CÓDIGO DE MANERA QUE SEA MÁS LEGIBLE Y PARA REUTILIZAR AQUELLAS PARTES QUE SE REPITEN A LO LARGO DE UNA APLICACIÓN.



RESUMEN Y RECOMENDACIONES

NO OLVIDEN INGRESAR SU ASISTENCIA EN

EL FORMS 😊



Ayudante: Myriam Alvarado

MUCHAS

GRACIAS

